MixerReturn user guide

MixerReturn is a plugin that **bolts a Dugan Automixer onto a console that hasn't got one**, post-fader, without spending an insert slot or a second channel strip per mic.

It comes as VST3, AU and Standalone, and gives a plugin host something it doesn't otherwise have: **a shared summing bus spanning many instances**. Put one instance in a rack fed from each automixed channel's rack, and one more set to output the sum, which goes back to the desk as a mix's External Input. That makes two racks per mic, because nothing can sit after the automixer in its own rack (see the signal flow).

> **Before you rely on this:** the summing bus is verified numerically against the actual shipped processor class (a one-block delay with the summing instance processed both first and last, 24 senders with the host's processing order reshuffled every block, and 400 blocks with 17 instances racing on their own threads), and it's clean under ThreadSanitizer. `pluginval` passes at strictness 8.
>
> It **has** been run with real audio in Waves SuperRack Performer v15.15.12 on macOS (48 kHz, 256-sample buffer), over a loopback virtual device rather than a desk. Audio went through the sum, and it was measured rather than listened to: the sum equals the sum of its senders delayed by exactly one block, with 0 of 143744 samples in error, pass-through is bit-exact, send trim and mute come back exact, and a bypassed instance contributes zero without wedging the bus. It **hasn't been run against a real console or used on a show**. **Every claim about SQ behaviour here comes from the reference guide, not from hardware.** Check it over before you use it on live gear.
>
> This codebase was created with AI assistance, directed and reviewed by a human author.

Why this exists

SuperRack's patcher allows **only one rack per output I/O**, so you can't patch twenty-four rack outputs onto one output to sum them. The desk can't sum them either, not without spending twenty-four input channels on it.

MixerReturn fills that one gap, and everything else about the design follows from it.

What it doesn't cost you

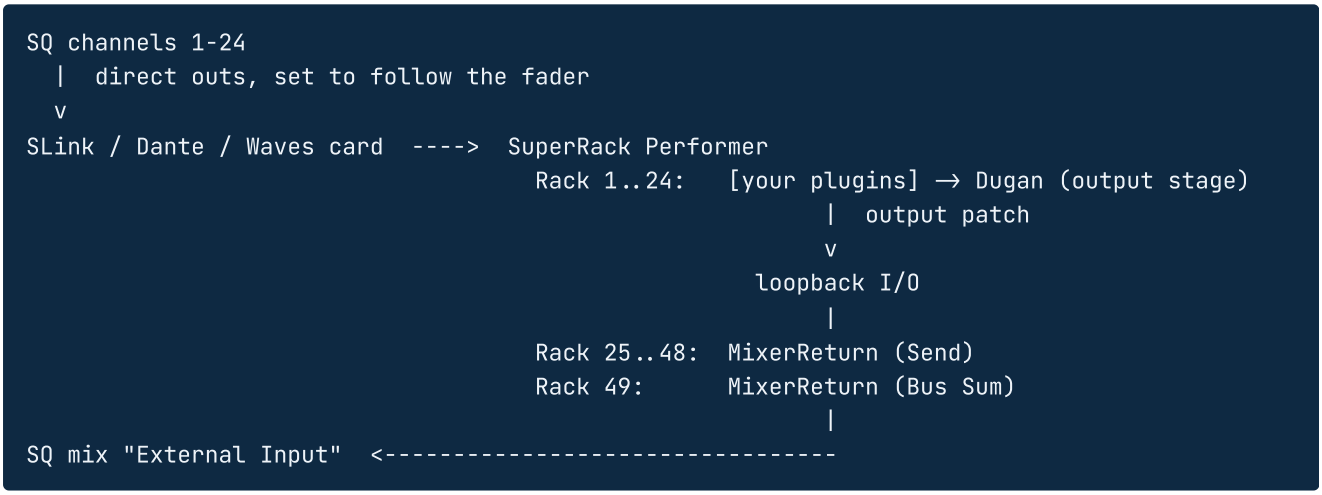
Because the automixer works on the **direct outs** rather than on the channel inserts:

- **The insert slots stay free.** You can still put plugins on the channel strips as normal, and the automixer isn't competing for them.
- **The automixer sits post-fader**, without moving any insert points.
- **No channel is used twice.** One channel strip per mic, as usual.

The signal flow

It's two racks per mic, not one. SuperRack's Dugan Automixer is part of a rack's *output stage*: it sits after all eight plugin slots, and there's no Dugan you can insert as a plugin. So nothing you put in a rack can be downstream of it, MixerReturn included. A MixerReturn sharing a rack with the Dugan sends the signal from *before* the automixer, and your return carries a plain sum with no automixing in it at all.

The fix is to let the first rack finish, and pick the automixed signal back up in a second:



The loopback is what this costs you: each Dugan rack's output has to reach a MixerReturn rack's input, which needs a spare I/O pair per mic on the machine's interface, or a virtual audio device that offers loopback ports.

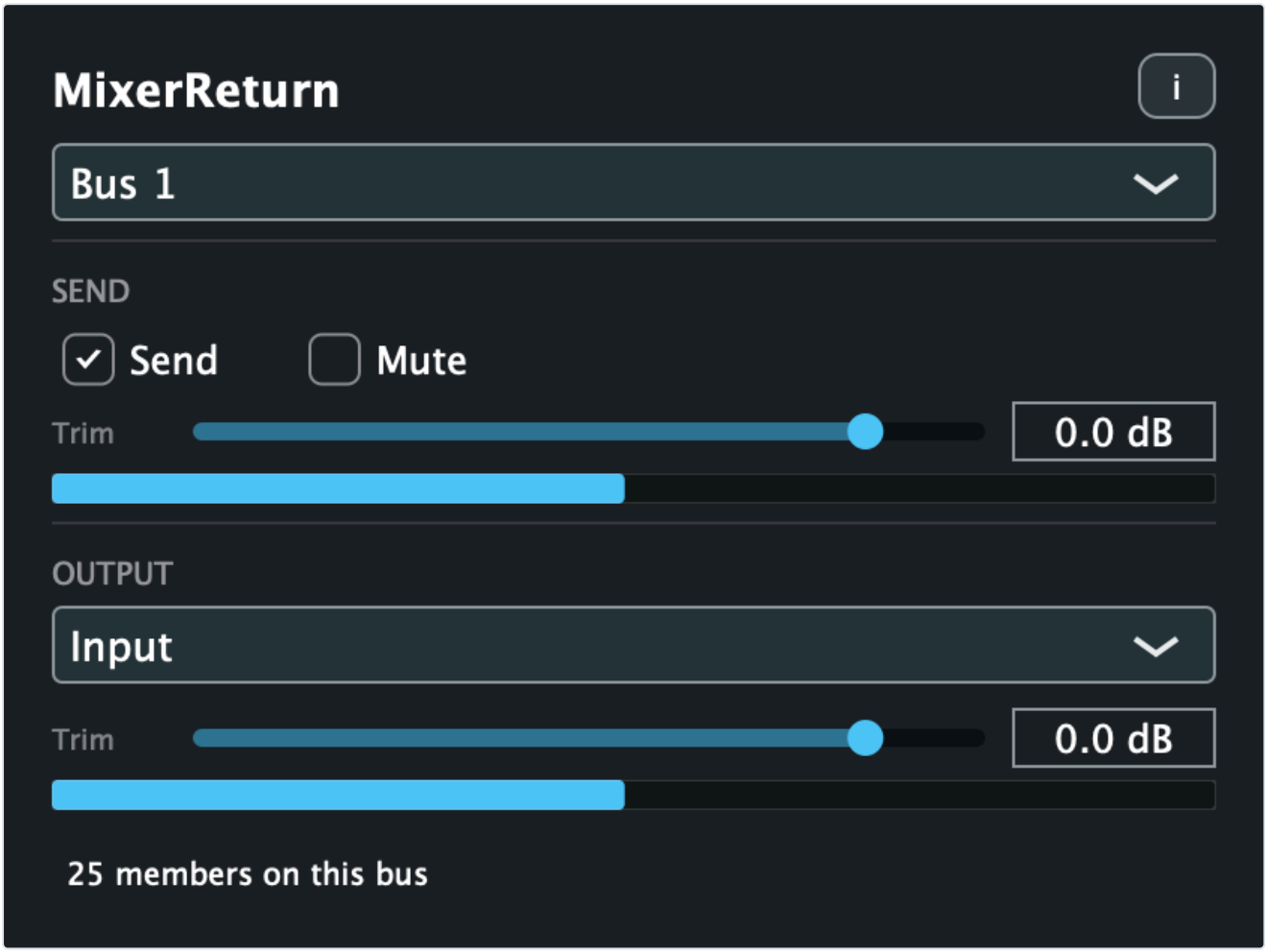
The operator keeps working ordinary channel strips, and the Dugan sees post-fader signals.

> **Why this is worth knowing before you patch anything.** You won't catch it by ear. With the automixer bypassed in the return path the sum still sounds like the mics, just without any gain sharing, which is what a plain sum sounds like. Measured on the bench with two channels open, the rack outputs were automixed to −4.39 dB while a same-rack MixerReturn's sum carried them at +0.02 dB, unattenuated.

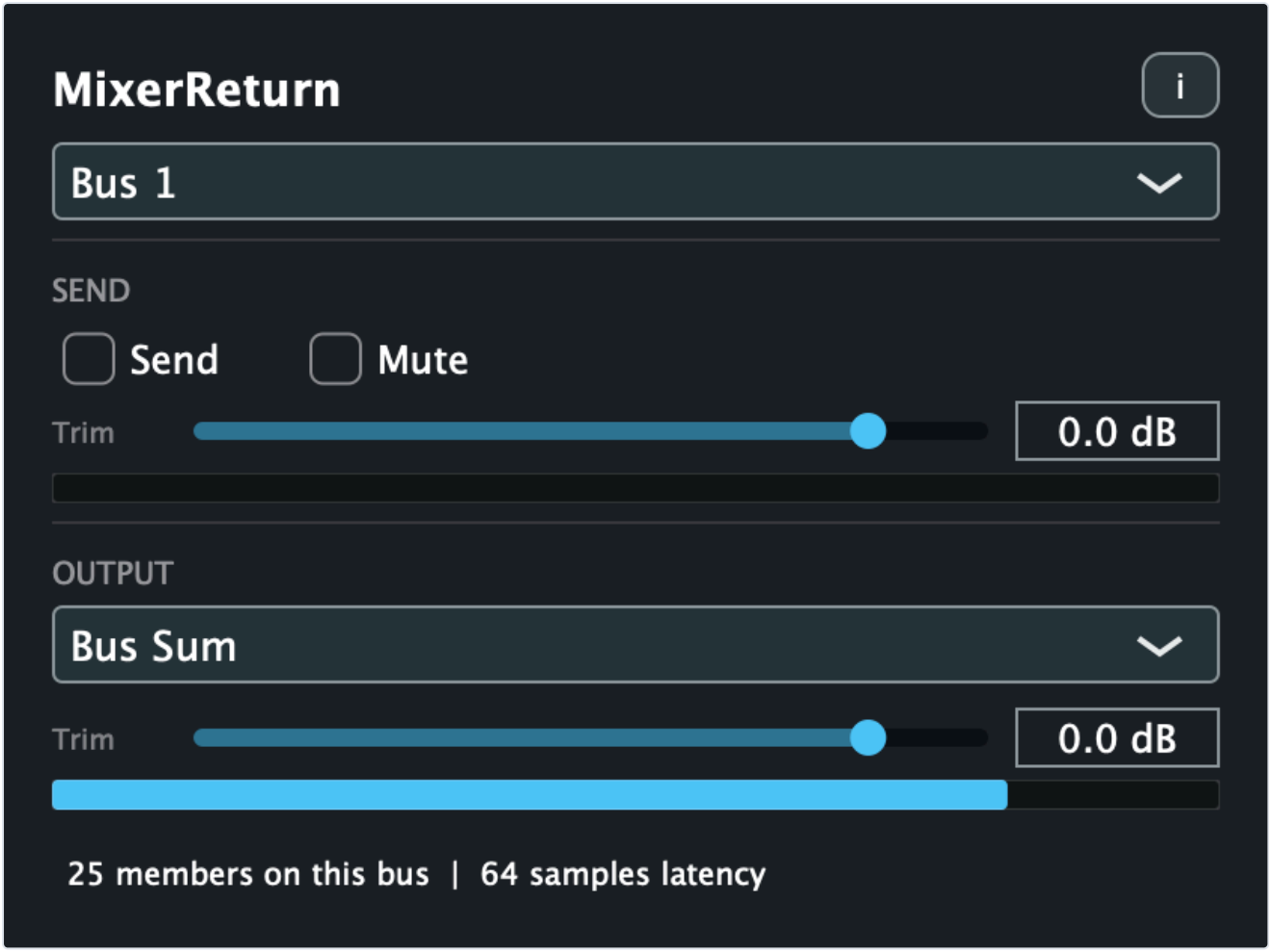
The two roles

Sending and receiving are independent, so an instance can be a sender, a receiver, or both.

On each MixerReturn rack (the second rack, fed from a Dugan rack's output): **Send** enabled, and **Output** set to `Input` so the rack passes its own audio through.



On the return rack: Send disabled, so the send meter is empty, and **Output** set to `Bus Sum`.



Both rendered from the actual editor with a full 24-channel rig registered on the bus, so the member count and latency readout are real values rather than a mock-up.

The **member count** is the readout to trust when you're setting up. If it doesn't say what you expect, an instance is on the wrong bus or hasn't loaded.

The controls

Control	What it does
Bus	Which of the 8 shared buses this instance joins.
Send / **Mute**	Whether this instance contributes its input to the bus.
Send Trim	−inf to +10 dB on the contribution, matching the SQ's own direct-out trim range.
Output	`Input` (pass through), `Bus Sum` (emit the sum), or `Input + Bus Sum`.
Output Trim	−inf to +10 dB on this instance's output.

The one thing to know

The bus sum is delayed by exactly one block, the same for every sender, whatever order the host processes the instances in.

That's on purpose, and it's why the design looks the way it does: a shorter but *uneven* delay across channels would comb-filter the sum. See DESIGN.md.

Setting it up on an Allen & Heath SQ

One detail on the SuperRack side and two on the console, and all three will bite you otherwise.

> **Budget two racks and a loopback pair per mic**, per "The signal flow" above. Twenty-four automixed mics means forty-eight racks plus a return rack, and twenty-four loopback pairs. Check the interface has the I/O before you commit to a channel count. This is the constraint most likely to decide how many mics you can automix on a given rig.

> **"Follow Fader" on direct outs is global across all 48 channels.** There's no post-fader tap point. Post-fader comes from that one switch, and it's all or nothing.
>
> **If you also multitrack off direct outs, that recording becomes post-fader too.** Route multitrack over **tie lines** (post-preamp, bypassing the processing path) instead.

> **Unroute the automixed channels from the main mix.** The summed return has to be their only path there. A channel reaching the mix both directly *and* through the return will comb.
>
> Pre-fader monitor sends stay inside the desk and are fine.

The SQ core runs at 96 kHz, so the plugin host has to as well.

Troubleshooting

Symptom	Cause
Member count is lower than expected	An instance is on a different bus, or hasn't loaded. The count is the authoritative readout.
Comb filtering on the automixed channels	They are reaching the main mix both directly and through the return. Unroute them from the mix.
Automixer is reacting to the wrong level	Direct outs are pre-fader. "Follow Fader" is one global switch on the SQ.
Multitrack recording levels now follow the faders	Same switch — it is global. Move multitrack to tie lines.
Bus Sum instance outputs nothing	Check **Output** is `Bus Sum` and not `Input`, and that senders are on the same bus.
Send meter is empty on a channel instance	**Send** is disabled, or **Mute** is on.
Sample-rate mismatch with the console	The SQ core is 96 kHz; the host must match.

See also

- DESIGN.md: the summing bus, and why the delay is uniform by design
- README: signal flow, controls and downloads

Installing

On macOS, install to the system folder, not your home folder:

```
/Library/Audio/Plug-Ins/VST3/
```

This matters more than it looks. **Waves SuperRack Performer only scans the system VST3 folder, and never looks in `~/Library/Audio/Plug-Ins/VST3/`.** A plugin dropped into the user folder just won't appear in SuperRack, with no error and nothing in a log to explain why. The `.pkg` installer puts it in the right place. If you use the `.zip`, copy the bundle to the path above rather than to your home folder.

That directory is world-writable on a stock macOS install, so you don't need an administrator password to copy into it.

SuperRack scans at launch only. If you install or update the plugin while SuperRack is running, quit and relaunch it before you expect to see the change.

You don't need to unquarantine anything. A locally built or `.zip`-extracted bundle carries no `com.apple.quarantine` attribute here, and the plugin loads and runs in SuperRack with an ad-hoc signature, so Developer ID signing and notarisation aren't needed for it to be hosted.

On Windows, the installer puts the VST3 in `C:\Program Files\Common Files\VST3\`, the folder every VST3 host scans, and the standalone in `C:\Program Files\MixerReturn\`. From the `.zip`, copy `VST3\MixerReturn.vst3` into that VST3 folder yourself. The v0.4.2 installer and older put the VST3 in `C:\Program Files\MixerReturn\VST3\` instead, where no host looks: move it across by hand, or run a newer installer, which clears that copy away.

Where this is going

The device is the main event. The plugin is the version that works without installing one.

Once the wrapper exists, a rack's output patch *is* the routing decision (a physical output to behave as an insert, or a `Sum` port to feed the buses), with no plugin in the chain at all. Technically the wrapper is better in every way: a driver gets every Sum port in one callback, so it can sum with **no added delay**, where the plugin has to cost a block. The whole two-page barrier in this plugin only exists because plugin instances can't see each other's timing, and a driver doesn't have that problem.

What the plugin still gives you, in full: it exists today. It needs no driver install, no admin rights, no notarised system extension and no signed kernel driver. It doesn't depend on successfully wrapping any particular interface, so it runs on locked-down or rented machines where a system audio device isn't an option. That makes it easier to deploy, but it doesn't do anything the device can't. The one capability gap runs the other way: routing between separate applications, which a per-process plugin bus can never do.
